HotelProject

현재 구현 상태를 바탕으로 한 다음 수업 방향 정리

결론 가장 먼저 할 일은 '여기어때와 같은 정보 구조를 참고한, 프로젝트 고유의 숙소 상세 페이지'를 완성하는 것입니다. 이후 DB 이미지, 지도, 검색 필터를 순서대로 연결합니다.

현재 프로젝트에서 이미 다룬 범위

프론트엔드	React Router로 메인과 /detail/:id 상세 경로를 구성하고, useEffect와 fetch로 API 데이터를 불러오는 흐름을 사용 중입니다.
백엔드	Spring Boot, JPA, 숙소 및 숙소 이미지 엔티티, DTO, 예외 처리 구조를 학습하고 있습니다.
화면 흐름	인기 숙소 목록에서 상세 페이지로 이동하는 기본 사용자 흐름이 있습니다.

권장 구현 순서

순서	주제	목표
1	상세 페이지 UI	사진 갤러리, 숙소 기본 정보, 편의시설, 객실 카드, 취소 규정, 위치 영역을 먼저 완성합니다.
2	DB 이미지 연결	숙소 상세 API가 DB의 이미지 URL 배열을 반환하고, 프론트 갤러리에 표시합니다.
3	지도 연결	주소 기반으로 지도와 마커를 표시하고, 나중에 위도·경도 컬럼으로 정확도를 높입니다.
4	검색 필터	지역·날짜·인원·유형 조건을 URL과 백엔드 검색 API로 연결합니다.
5	성능 측정	실제 API와 데이터가 쌓인 뒤 AOP로 실행 시간과 개선 전후를 비교합니다.
6	RAG 추천	숙소 설명·리뷰·주변 정보가 충분히 쌓였을 때 자연어 추천 기능으로 확장합니다.

상세 페이지 설계 방향

참고 이미지의 정보 밀도와 예약 흐름을 활용하되, 특정 서비스의 브랜드·문구·레이아웃을 복제하지 않고 HotelProject만의 디자인으로 만듭니다.

사진 갤러리	대표 이미지 1장과 보조 이미지 4장으로 시작합니다. 이미지 수, 전체 사진 보기 버튼, 이미지가 부족할 때의 대체 화면을 포함합니다.
숙소 기본 정보	숙소 유형, 이름, 평점, 리뷰 수, 주소, 찜 버튼과 할인 전후 가격을 분명하게 보여줍니다.
핵심 안내 카드	후기 요약, 주차·조식·환불 여부 같은 편의시설, 위치 요약을 한눈에 제시합니다.
객실 선택	객실 사진, 객실명, 체크인/아웃 시간, 취소 정책, 가격, 예약 버튼을 카드 형태로 구성합니다. 초기에는 목업 데이터여도 이후 API로 바꾸기 쉬운 구조가 좋습니다.
하단 정보	숙소 소개, 취소 및 환불 규정, 지도와 주변 정보를 배치합니다. 데스크톱에서는 예약 요약 영역을 고정해도 좋습니다.

구현 시 함께 챙길 상태

로딩 중 화면, API 오류 화면, 숙소를 찾지 못한 404 화면, 이미지 배열이 비어 있는 경우의 대체 이미지를 상세 페이지의 기본 상태로 포함합니다.

상세 페이지 구현 프롬프트

현재 React + Vite 숙소 예약 프로젝트의 Detail 페이지를 개선해줘.

목표: 국내 숙소 예약 서비스처럼 신뢰감 있고 깔끔한 상세 페이지를 만든다. 기존 React Router의 /detail/:id 구조와 fetch API 연동은 유지한다. 특정 서비스의 로고·문구·디자인을 그대로 복제하지 않고, 독창적인 예약 서비스 디자인으로 구현한다. 반응형 디자인을 적용한다.

구성: 대표 이미지 1장과 보조 이미지 4장의 갤러리, 숙소 유형·이름·평점·리뷰 수·주소·찜 버튼, 할인 전후 가격, 후기·편의시설·위치 요약 카드, 객실 사진·객실명·체크인/아웃·취소 규정·가격·예약 버튼이 있는 객실 카드, 숙소 소개·취소 규정·위치 섹션을 만든다. 지도 API 연결 전에는 지도 placeholder를 사용한다.

상태 처리: API 로딩, 오류, 404, 이미지 배열이 비었을 때의 대체 화면을 만든다.

품질: 역할별 컴포넌트로 적절히 분리하고, 불필요한 inline style은 CSS로 옮긴다. useEffect 의존성을 올바르게 처리하고, 이미지 alt와 버튼 aria-label을 제공한다.

지도, 검색, RAG, 성능 측정의 역할

기능을 너무 일찍 확장하기보다, 실제 사용자 흐름과 데이터 기반을 먼저 만들면 각 기술이 필요해지는 이유가 분명해집니다.

지도

상세 페이지 UI가 완성된 뒤 연결합니다. 현재 address만으로도 주소 검색 기반 지도를 표시할 수 있고, 이후 latitude·longitude 컬럼을 추가하면 마커 위치를 더 정확히 제어할 수 있습니다. 국내 서비스는 카카오맵 또는 네이버맵이 잘 맞습니다.

검색 필터

DB에서 처리하는 것이 맞습니다. 메인에서 지역·날짜·인원을 입력하고, URL 쿼리스트링을 통해 검색 결과 페이지로 이동합니다. 백엔드는 RequestParam으로 조건을 받고 JPA 조회로 결과를 반환합니다.

RAG

지금은 우선순위가 낮습니다. 데이터가 적은 초기에는 SQL 필터(지역·가격·유형·평점·편의시설)만으로도 좋은 추천을 만들 수 있습니다. 숙소 설명, 리뷰, 주변 관광 정보가 충분히 쌓인 뒤 '조용한 바다 근처 숙소 추천' 같은 자연어 추천 기능으로 확장합니다.

AOP 성능 측정

좋은 학습 주제지만 검색·상세 API와 데이터가 실제로 갖춰진 다음이 적절합니다. API별 실행 시간, DB 쿼리 수, 이미지 조회 방식 개선 전후를 비교하면 최적화의 근거를 만들 수 있습니다.

최종 사용자 흐름

메인 필터 입력	검색 결과 페이지	백엔드 조건 검색 API	DB 조건 조회	숙소 카드	상세 페이지

현재 결론: 상세 UI → DB 이미지 → 지도 → 검색 필터 → AOP 성능 측정 → RAG 추천 순서로 진행합니다.